SQL Joins: Visual Reference Guide

RELATIONAL ALGEBRA, SET THEORY & EVALUATED RESULTS

Reference Manual • Part 1

A **JOIN** matches rows across multiple tables via related keys. The reference schema below includes `dept_id` (foreign key matching Departments) and `manager_id` (self-referencing unary key matching another Employee):

ACTIVE WORKING SCHEMA & ROW RECORDS

Table A: `Employees` (4 rows)

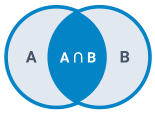
emp_id	name	dept_id	manager_id
1	Alice	10	*NULL* (CEO)
2	Bob	20	1 (Alice)
3	Charlie	30	1 (Alice)
4	David	*NULL*	2 (Bob)

Table B: `Departments` (3 rows)

dept_id	dept_name
10	Engineering
20	Marketing
40	Finance

1. INNER JOIN

Strict Intersection (A ∩ B)



Common Keys Only

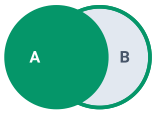
Returns rows where matching values exist in **both tables**. Rows with unreferenced keys (e.g., Charlie dept 30, David dept NULL, Finance dept 40) are strictly excluded.

```
SELECT E.name, D.dept_name FROM Employees E INNER JOIN Departments D ON E.dept_id = D.dept_id;
```

name	dept_name	Relational Evaluation
Alice	Engineering	dept_id 10 matched in both tables
Bob	Marketing	dept_id 20 matched in both tables

2. LEFT JOIN (LEFT OUTER JOIN)

Complete Left Set + Matched Right



All Table A Preserved

Preserves **all records from Table A**. For rows without a corresponding match in Table B, right-hand attributes are populated with NULL.

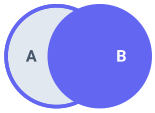
```
SELECT E.name, D.dept_name FROM Employees E LEFT JOIN Departments D ON E.dept_id = D.dept_id;
```

name	dept_name	Relational Evaluation
Alice	Engineering	Matched (dept 10)
Bob	Marketing	Matched (dept 20)
Charlie	*NULL*	dept 30 has no entry in Table B
David	*NULL*	dept_id is NULL (unassigned)

3. RIGHT JOIN (RIGHT OUTER JOIN)

Complete Right Set + Matched Left

Preserves **all records from Table B**, regardless of whether a matching employee exists in Table A. Unmatched left-side attributes are assigned NULL.



All Table B Preserved

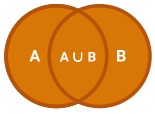
```
SELECT E.name, D.dept_name FROM Employees E RIGHT JOIN Departments D ON E.dept_id = D.dept_id;
```

name	dept_name	Relational Evaluation
Alice	Engineering	Matched with emp_id 1
Bob	Marketing	Matched with emp_id 2
NULL	Finance	dept 40 exists in B, but has 0 assigned staff

4. FULL OUTER JOIN (FULL JOIN)

Complete Union with Null Padding (A ∪ B)

Yields all records from both datasets. If there is no matching pair, the missing side produces NULL values, leaving no record omitted.



Complete Bi-Directional Set

```
SELECT E.name, D.dept_name FROM Employees E FULL OUTER JOIN Departments D ON E.dept_id = D.dept_id;
```

name	dept_name	Relational Evaluation
Alice	Engineering	Matched pair (dept 10)
Bob	Marketing	Matched pair (dept 20)
Charlie	*NULL*	Left unmatched (dept 30 absent in B)
David	*NULL*	Left unmatched (dept_id is NULL)
NULL	Finance	Right unmatched (dept 40 has no employees)

SPECIALIZED PATTERN: ANTI-JOINS (FINDING MISSING OR ORPHAN RECORDS)

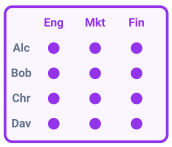
An *Anti-Join* isolates rows from one table that have **no corresponding record** in the other. It is authored by adding a `WHERE [Right_Key] IS NULL` condition to an outer join.

```
SELECT E.name FROM Employees E LEFT JOIN Departments D ON E.dept_id = D.dept_id WHERE D.dept_id IS NULL; -- Output: Charlie, David (Employees lacking valid department assignments)
```

Cartesian Products & Hierarchical Self Joins

5. CROSS JOIN (Cartesian Product)

Combinatorial Product (M × N)



4 rows × 3 rows = 12 rows

Pairs **every row in Table A** with **every row in Table B**. It requires no join condition (ON clause). Used for permutations, inventory matrices, and schedule grids.

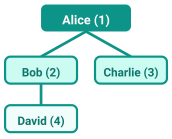
```
SELECT E.name, D.dept_name FROM Employees E CROSS JOIN Departments D;
```

#	name	dept_name
1	Alice	Engineering
2	Alice	Marketing
3	Alice	Finance
4	Bob	Engineering
5	Bob	Marketing
6	Bob	Finance

#	name	dept_name
7	Charlie	Engineering
8	Charlie	Marketing
9	Charlie	Finance
10	David	Engineering
11	David	Marketing
12	David	Finance

6. SELF JOIN (Unary Hierarchy)

Table Joined with Itself



Reporting Hierarchy

A table joined to **itself** using aliases (`E` for employee, `M` for manager). Matches each employee's `manager_id` to another row's `emp_id`.

```
SELECT E.name AS Employee, M.name AS Manager FROM Employees E LEFT JOIN Employ
ees M ON E.manager_id = M.emp_id;
```

Employee	Manager	Structural Hierarchy Relationship
Alice	*NULL*	Top-level Executive / CEO (manager_id is NULL)
Bob	Alice	Direct report to Alice (manager_id = 1)
Charlie	Alice	Direct report to Alice (manager_id = 1)
David	Bob	Direct report to Bob (manager_id = 2)

SQL JOIN SELECTION QUICK GUIDE

- **INNER JOIN:** Use when you only want complete, fully validated records matching across both sides.
- **LEFT JOIN:** Default choice for entity reports where child records might not exist yet (e.g., Customers with or without Orders).
- **CROSS JOIN:** Generates combinatorial matrix sets; ensure datasets are small to avoid explosive memory usage.
- **SELF JOIN:** Essential for graphs, trees, nested category tags, and multi-tier organizational charts within one table.